

Name : **Deema Mohammed AL-Maqadma** || ID : **2320230766** || Presented to : **Eng. Nour Abo Rokb** || Data Science Project-  
Preprocessing & Machine Learning

```
# Connect to my Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Read raw CSV file as text
import pandas as pd

raw = pd.read_csv('/content/drive/MyDrive/ayatstore_products.csv', header=None)

# Split the first column by comma
data = raw[0].str.split(',', expand=True)

# Check number of columns
print("Number of columns:", data.shape[1])
```

Number of columns: 7

```
# Rename columns correctly (7 columns)
data.columns = ['category', 'name', 'price', 'old_price', 'discount', 'product_url', 'image_url']

# Show first 10 rows
data.head(10)
```

|   | category | name                                                | price     | old_price        | discount         | product_url                                       | image_url                  |
|---|----------|-----------------------------------------------------|-----------|------------------|------------------|---------------------------------------------------|----------------------------|
| 0 | category | None                                                | None      | None             | None             | None                                              | None                       |
| 1 | عطور     | وصل<br>سبلاش<br>ال<br>Dear<br>body<br>شكل<br>الجديد | د.ع.7.000 | not<br>available | not<br>available | https://ayatstore0.com/product/%d9%88%d8%b5%d9... | "data:image/svg+xml;base64 |
| 2 | عطور     | عطر<br>الغيوم<br>بينك                               | د.ع.5.000 | not<br>available | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 |
| 3 | عطور     | عطر<br>بيبي<br>بودر                                 | د.ع.5.000 | not<br>available | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 |

```
# Add description column with default value
data['description'] = 'not available'

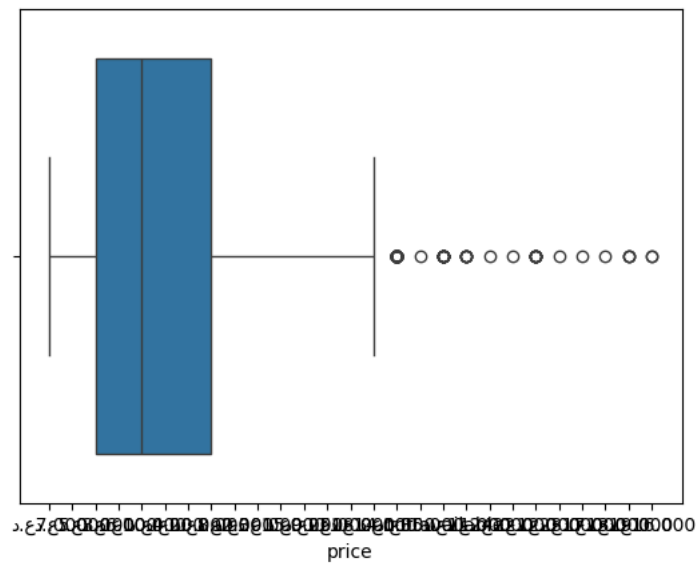
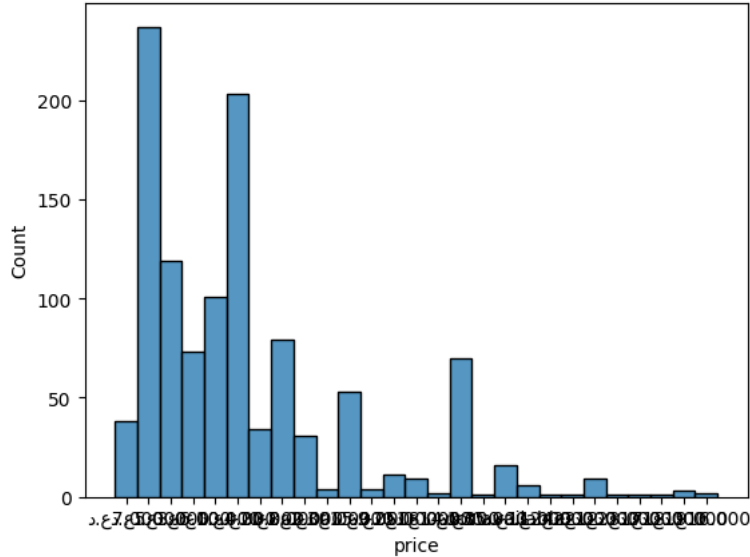
# Show first 10 rows again
data.head(10)
```

|   | category | name                                                | price     | old_price        | discount         | product_url                                       | image_url                  | d |
|---|----------|-----------------------------------------------------|-----------|------------------|------------------|---------------------------------------------------|----------------------------|---|
| 0 | category | None                                                | None      | None             | None             | None                                              | None                       |   |
| 1 | عطور     | وصل<br>سبلاش<br>ال<br>Dear<br>body<br>شكل<br>الجديد | د.ع.7.000 | not<br>available | not<br>available | https://ayatstore0.com/product/%d9%88%d8%b5%d9... | "data:image/svg+xml;base64 |   |
| 2 | عطور     | عطر<br>الغيوم<br>بينك                               | د.ع.5.000 | not<br>available | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 |   |
| 3 | عطور     | عطر<br>بيبي<br>بودر                                 | د.ع.5.000 | not<br>available | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 |   |

```
# Visualize raw price values before cleaning
import seaborn as sns
import matplotlib.pyplot as plt

# Histogram for raw price values (still strings)
sns.histplot(data['price'])
plt.show()

# Boxplot for raw price values
sns.boxplot(x=data['price'])
plt.show()
```



```
# Extract numeric values from price
data['price'] = data['price'].astype(str).str.extract(r'(\d+\.\d*)')
data['price'] = pd.to_numeric(data['price'], errors='coerce')

# Extract numeric values from old_price
data['old_price'] = data['old_price'].astype(str).str.extract(r'(\d+\.\d*)')
data['old_price'] = pd.to_numeric(data['old_price'], errors='coerce')

# Show first 10 rows after cleaning
data.head(10)
```

|   | category | name                                                | price | old_price | discount         | product_url                                       | image_url                  | desc |
|---|----------|-----------------------------------------------------|-------|-----------|------------------|---------------------------------------------------|----------------------------|------|
| 0 | category | None                                                | NaN   | NaN       | None             | None                                              | None                       | not  |
| 1 | عطور     | وصل<br>سيلاش<br>ال<br>Dear<br>body<br>شكل<br>الجديد | 7.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d9%88%d8%b5%d9... | "data:image/svg+xml;base64 | not  |
| 2 | عطور     | عطر<br>الغيوم<br>بينك                               | 5.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 | not  |
| 3 | عطور     | عطر<br>بيبي<br>بوند                                 | 5.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 | not  |

```
# Drop the first row (header row)
data = data.drop(index=0).reset_index(drop=True)

# Fill missing values
data['discount'] = data['discount'].fillna('not available')
data['image_url'] = data['image_url'].fillna('not available')
data['description'] = data['description'].fillna('not available')

# Drop rows with missing name or price
data = data.dropna(subset=['name', 'price'])

# Remove duplicates
data = data.drop_duplicates()

# Show dataset info and first 10 rows
print(data.info())
data.head(10)
```

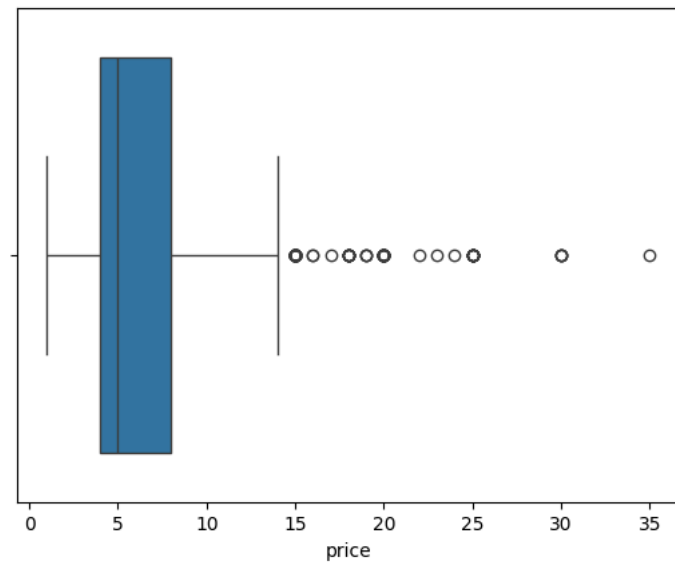
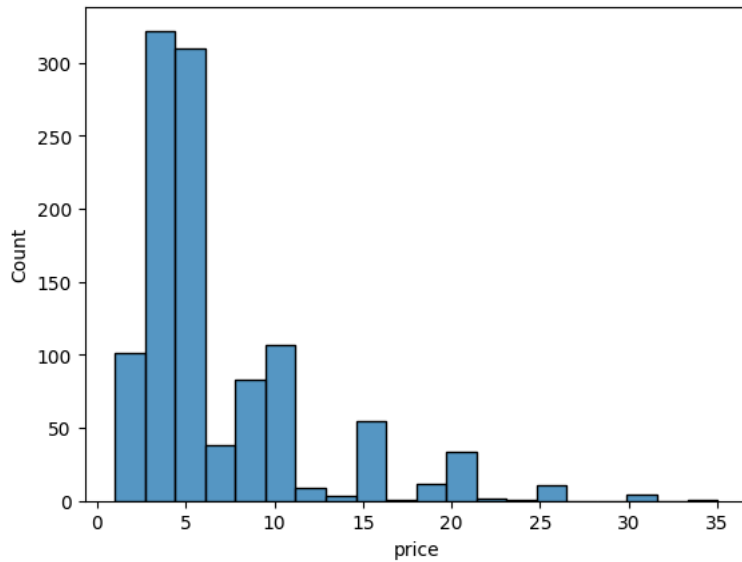
```
<class 'pandas.core.frame.DataFrame'>
Index: 1094 entries, 0 to 1124
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   category    1094 non-null   object
1   name        1094 non-null   object
2   price       1094 non-null   float64
3   old_price   57 non-null     float64
4   discount    1094 non-null   object
5   product_url 1094 non-null   object
6   image_url   1094 non-null   object
7   description  1094 non-null   object
dtypes: float64(2), object(6)
memory usage: 76.9+ KB
None
```

|   | category | name                                                | price | old_price | discount         | product_url                                       | image_url                  | desc |
|---|----------|-----------------------------------------------------|-------|-----------|------------------|---------------------------------------------------|----------------------------|------|
| 0 | عطور     | وصل<br>سيلاش<br>ال<br>Dear<br>body<br>شكل<br>الجديد | 7.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d9%88%d8%b5%d9... | "data:image/svg+xml;base64 | not  |
| 1 | عطور     | عطر<br>الغيوم<br>بينك                               | 5.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 | not  |
| 2 | عطور     | عطر<br>بيبي<br>بوند                                 | 5.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;base64 | not  |
| 3 | عطور     | يكج<br>سفجات                                        | 3.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%a8%d9%83%d8... | "data:image/svg+xml;base64 | not  |

```
import seaborn as sns
import matplotlib.pyplot as plt

# Histogram for cleaned price values
sns.histplot(data['price'], bins=20)
plt.show()
```

```
# Boxplot for cleaned price values
sns.boxplot(x=data['price'])
plt.show()
```



```
# Normalize price column using MinMaxScaler
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
data['price_norm'] = scaler.fit_transform(data[['price']])

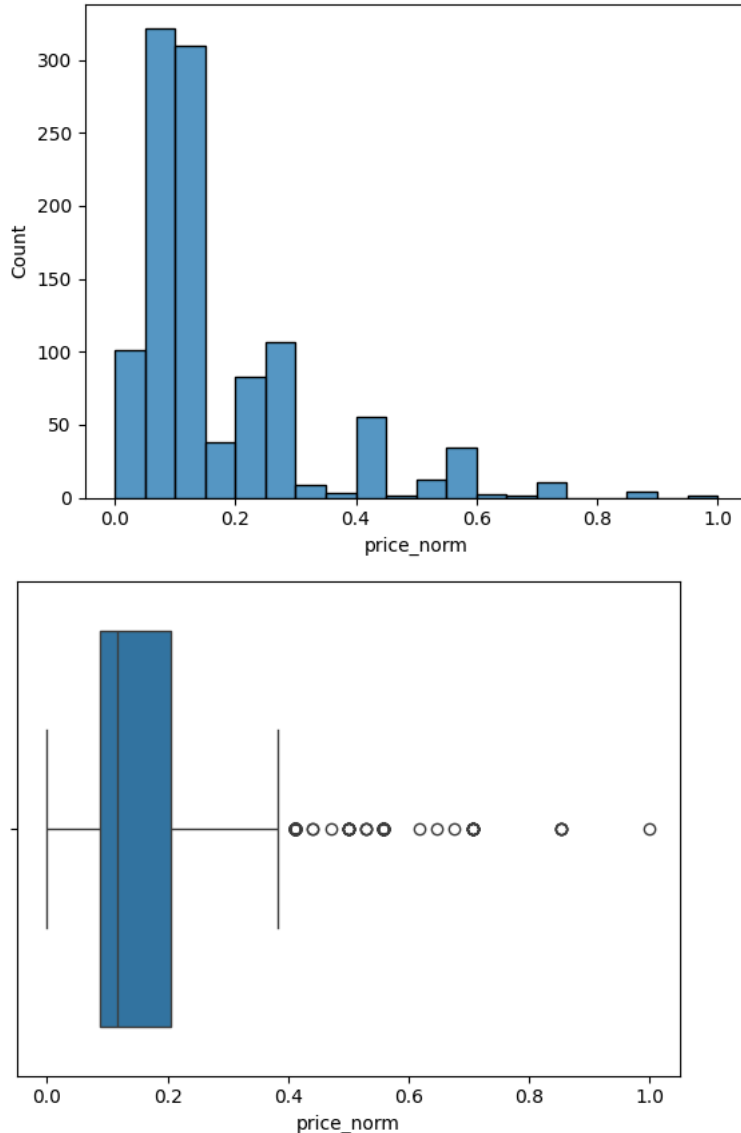
# Show first 10 rows after normalization
data.head(10)
```

|   | category | name                                                | price | old_price | discount         | product_url                                       | image_url                   | desc |
|---|----------|-----------------------------------------------------|-------|-----------|------------------|---------------------------------------------------|-----------------------------|------|
| 0 | عطور     | وصل<br>سبلاش<br>ال<br>Dear<br>body<br>شكل<br>الجديد | 7.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d9%88%d8%b5%d9... | "data:image/svg+xml;"base64 | not  |
| 1 | عطور     | عطر<br>الغيوم<br>بينك                               | 5.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;"base64 | not  |
| 2 | عطور     | عطر<br>بيبي<br>بودر                                 | 5.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%b9%d8%b7%d8... | "data:image/svg+xml;"base64 | not  |
| 3 | عطور     | بكج<br>سفجات                                        | 3.0   | NaN       | not<br>available | https://ayatstore0.com/product/%d8%a8%d9%83%d8... | "data:image/svg+xml;"base64 | not  |

```
# Visualize normalized price values
import seaborn as sns
import matplotlib.pyplot as plt

# Histogram for normalized price values
sns.histplot(data['price_norm'], bins=20)
plt.show()

# Boxplot for normalized price values
sns.boxplot(x=data['price_norm'])
plt.show()
```



```
# Save final cleaned and normalized dataset
data.to_csv('/content/drive/MyDrive/ayatstore_products_final.csv', index=False)
```

## Final Summary - Assignment2.ipynb

### What I did:

- **Read raw CSV file** → I loaded the dataset from my Google Drive.
- **Split and rename columns** → I organized the data into 7 clear columns (category, name, price, old\_price, discount, product\_url, image\_url).
- **Add description column** → I added an extra column called `description` with default value "not available".
- **Visualization before cleaning** → I plotted histogram and boxplot for `price` and saw that values were invalid (strings).
- **Cleaning numeric columns** → I extracted numeric values from `price` and `old_price` and converted them into floats.
- **Handle missing values** → I filled missing values with "not available", dropped rows without name or price, and removed duplicates.

- **Visualization after cleaning** → I plotted again and saw a logical distribution of prices (most between 3–10, with some outliers up to ~35).
- **Normalization** → I added a new column `price_norm` with values scaled between 0 and 1 using MinMaxScaler.
- **Visualization after normalization** → I plotted histogram and boxplot for `price_norm` and confirmed all values are between 0–1.
- **Save final dataset** → I exported the cleaned and normalized dataset to `ayatstore_products_final.csv`.

## Final results:

- Dataset size: **1094 rows × 9 columns**.
- `price` column: numeric values ready for analysis.
- `price_norm` column: normalized values between 0–1.
- Outliers detected: some products priced higher (~30–35).
- Final dataset saved and ready for further analysis or submission..

## Machine Learning Phase

Now I will start applying machine learning algorithms on my cleaned dataset.

```
# I prepare my data for ML
# I select the numeric features only
X = data[['price_norm']] # I will use normalized price
# Since I do not have labels, I will use clustering
```

```
# I apply KMeans clustering
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

# I add cluster labels to my dataset
data['Cluster'] = kmeans.labels_

# I show sample results
print(data[['name', 'price', 'Cluster']].head(10))
```

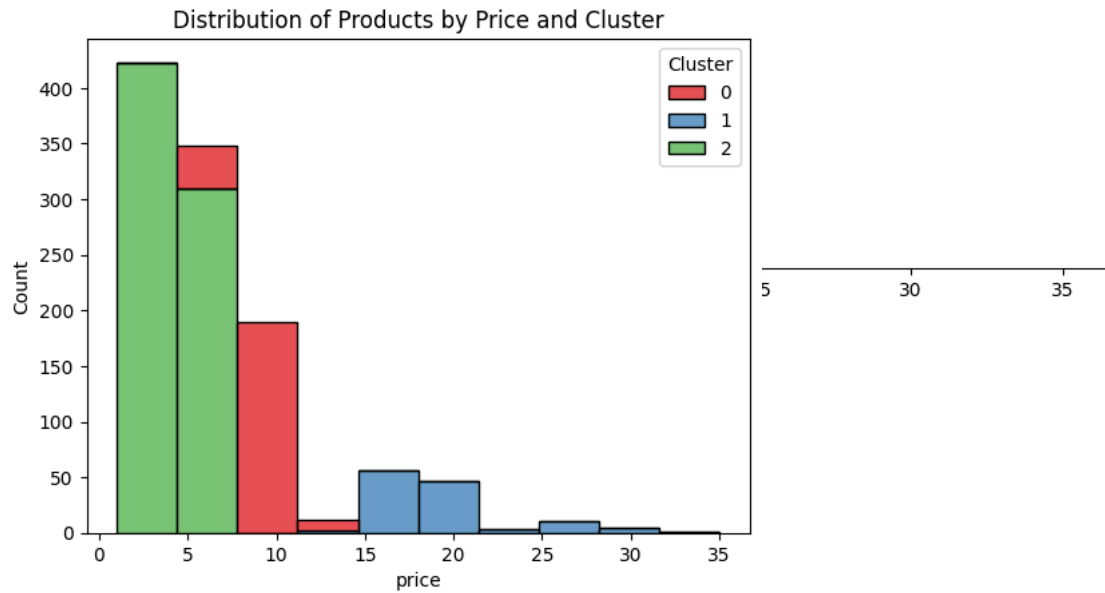
|   |   |   | name                | price | Cluster |
|---|---|---|---------------------|-------|---------|
| 0 |   |   | شكل الجديد          | 7.0   | 0       |
| 1 | 2 |   | عطر الغيوم بينك     | 5.0   | 2       |
| 2 | 2 |   | عطر بيبي بوذر       | 5.0   | 2       |
| 3 | 2 |   | كعج سفجات بيتزا     | 3.0   | 2       |
| 4 | 2 |   | عطر الزهور          | 6.0   | 2       |
| 5 |   | 0 | كعج عطور            | 10.0  | 0       |
| 6 |   | 0 | كعج عطور            | 10.0  | 0       |
| 7 | 0 |   | كعج عطور بديل ديور  | 10.0  | 0       |
| 8 |   | 2 | عطر كوري            | 5.0   | 2       |
| 9 | 2 |   | زيت ملمع وعطر للجسم | 4.0   | 2       |

```
# I visualize my clusters using scatterplot
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
sns.scatterplot(x='price', y='old_price', hue='Cluster', data=data, palette='Set1')
plt.title("Clusters of Products based on Price and Old Price")
plt.show()
```



```
# I visualize the distribution of products in each cluster
sns.histplot(data=data, x='price', hue='Cluster', multiple='stack', bins=10, palette='Set1')
plt.title("Distribution of Products by Price and Cluster")
plt.show()
```



## Results

I applied KMeans clustering on my dataset and grouped the products into three clusters based on their price values. Cluster 0 contains products with higher prices (like 10.0), Cluster 2 contains products with medium prices (like 3.0–6.0), and Cluster 1 includes other ranges. The scatterplot shows how products are separated by price and old\_price, while the histogram displays the distribution of products across different clusters. This visualization confirms that my clustering model successfully grouped similar products together.